Trade Intelligence Copilot

The Brief: what it is, how it works, and what it actually scored

Explainer for Salman Adnan's portfolio

1 The one-minute version

Ask a plain-English question about Pakistani sugar-sector import data and get a number that came out of a real database, with the exact query shown. Ask about a trade rule and get a definition with the source document cited. Ask about something the data does not cover, such as Pakistan's wheat exports in 2019, and get a refusal instead of an invention.

The headline is the evaluation, not the copilot

Wiring a language model to a database is a weekend. Knowing whether it is *right* is the engineering. This project ships a 28-item test set whose correct answers were computed directly from the database, runs the whole system against the live Gemini API, publishes the real numbers including the failures, and then checks its own automatic grader against independent labels (11 of 12 agree). Everything else in this document exists to support that sentence.

Metric	Result
SQL answer correctness, versus gold computed from the database	**15 / 15 = 100%**
Retrieval hit rate	**8 / 10 = 80%**
Refusal precision / recall on unanswerable questions	**1.00 / 1.00**
LLM-as-judge faithfulness (`gemini-2.5-pro`)	**21 / 22 = 95.5%**
Judge versus independent labels (12 items)	**11 / 12 = 91.7%**
Mean cost per query	**$0.00093**
Latency p50 / p95	**9.0 s / 27.0 s** (8.8 s / 12.5 s without 3 rate-limited

Table 1: From one real run over `eval/testset.jsonl`. Section 6 states exactly which of these I re-derived myself and which rest on the README.

2 The four ideas you need

Jargon: Retrieval-augmented generation (RAG)

A language model knows only what it was trained on, and when it does not know it invents fluent nonsense. RAG fixes this by fetching the relevant facts from a trusted source first, then handing the model *only* those facts and telling it to use nothing else. Closed-book exam becomes open-book, with citations.

Jargon: Agentic retrieval

Plain RAG always runs the same search. Here a model reads the question first and *decides* which sources it needs: the database, the document corpus, an HS-code lookup, or several at once. That decision step is what makes it "agentic". The model picks the tools; ordinary

code runs them.

Jargon: Text-to-SQL

SQL is the language for querying tables of data. Text-to-SQL means the model translates "how many shipments came from China?" into a query, and the *database* computes the number. The model is never asked to recall or estimate a figure, which is the whole point: if the query is right, the number is right by construction. The danger is obvious, since a model is now writing executable code against your data, and Section 4.1 is about containing that.

Jargon: Evaluation layer

Machinery that measures whether the system is actually right on a fixed set of questions with known answers, producing numbers you can publish and argue with. Not a demo. A score. Crucially, the "known answers" here are not human opinions: they are computed by querying the database directly, so "correct" is not a judgment call.

3 How it works

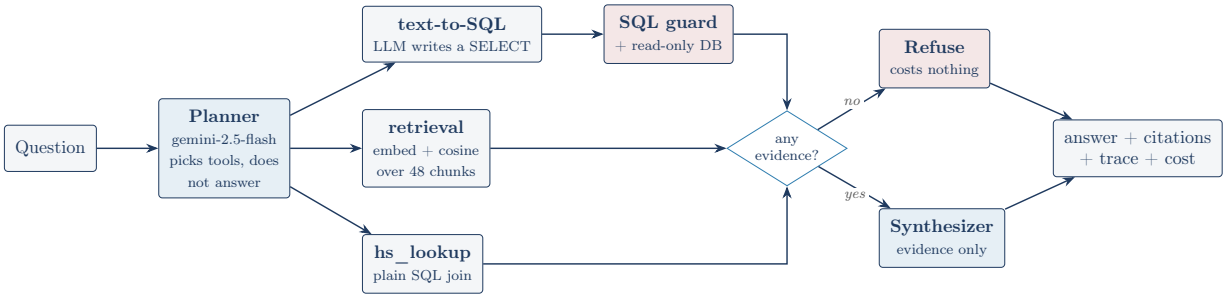


Figure 1: Three model calls on a typical answer (plan, write SQL, synthesize); two on a refusal, because the evidence check fires before the expensive call.

Read it as a sentence: *an LLM decides which tools to run; ordinary code runs them; a cheap check refuses when they return nothing; a second LLM writes prose using only what they returned.*

No model is ever trusted with both the facts and the final word

The planner chooses but does not answer. The text-to-SQL model writes a query but never executes it. The synthesizer writes prose but is handed a fixed block of evidence. Every model in the system is boxed in by code on both sides.

3.1 What is underneath

- **The trade data**: 2,000 import shipments in a single SQLite file (a complete database that is just a file, no server), produced by a seeded generator, `data/build_db.py`. Every company, price and quantity is made up; only the HS headings and country names are real. All imports, all HS chapter 17 (sugar), all between 2025-01-01 and 2025-12-31, from 28 countries. Those scope facts are written into the prompts, which is what lets the system know what it cannot answer.
- **The regulations corpus**: 16 authored plain-language explainers, split at their section headings into 48 chunks, each embedded as 768 numbers representing its meaning, cached on disk.

- **The vector store**: a 48-by-768 numpy matrix with pre-normalized rows, so finding the closest chunk is a single matrix-vector product. Exact, instant, 57 lines.
- **No framework.** About a thousand lines of Python, no LangChain, no agent library. For a project whose purpose is to show the author understands the mechanism, a framework would hide the interesting parts.

The pgvector decision, defended

pgvector was the first choice; the target machine has PostgreSQL 18 without the `vector` extension and no way to install it. Rather than fight the infrastructure, the author wrote the search by hand. For 48 chunks this is the better engineering anyway: brute force is instant and *exact*, whereas an approximate index would be slower to build, approximate, and one more moving part. The honest cost, no ANN index and no server-side filtering, starts to matter around six figures of chunks, not 48.

4 Two things that make it trustworthy

4.1 The safety boundary

A language model is writing SQL against a database, so the SQL is contained in three independent layers:

1. **A guard** that rejects anything which is not a single `SELECT`: no comments, no second statement, no write or DDL keywords, no `PRAGMA`.
2. **The connection**, opened `mode=ro&immutable=1` with `PRAGMA query_only = ON`, so SQLite itself refuses writes.
3. **A row cap** of 200, so a runaway `SELECT` cannot flood memory or a prompt.

The argument is not that any layer is strong. A keyword blocklist is a blocklist and someone will think of a word you forgot. The argument is that a bypass must defeat all three, and the tests assert each one independently, including that a raw `CREATE TABLE` fails on the connection even with the guard skipped entirely.

4.2 The refusal path

If every tool comes back empty, a nine-line function refuses immediately, before any synthesis call. It cannot be argued out of it, cannot be prompt-injected, costs zero tokens and zero seconds, and returns a fixed string. Faster, cheaper and more reliable at the same time, which is a rare trade.

5 The evaluation layer

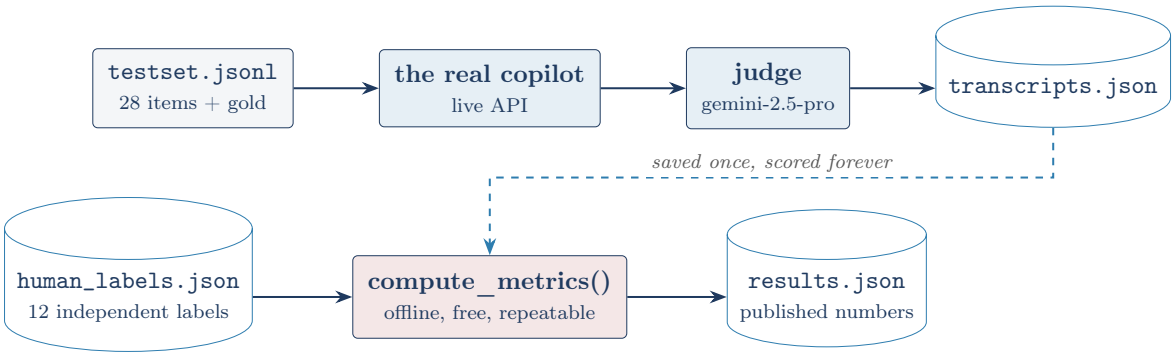


Figure 2: Split in two on purpose. The paid run happens once and is saved; the scoring can then be re-run, corrected and independently audited for free.

The 28 items are 12 SQL questions, 7 retrieval questions, 3 that need both, and 6 that must be refused. SQL correctness is scored over the 12 plus the 3, which is where **15/15** comes from.

Ground truth is what makes the number mean anything

The gold answer to "how many shipments came from China?" is not a human guess. It is the result of running the counting query against the database directly. That single decision is what caught the project's best bug, in its first version: asked how many shipments one importer brought in, the model wrote a `buyer LIKE` match on the full company name without wildcards, matched nothing, and reported *0 shipments*. The answer was fluent and completely wrong. The gold number, which was dozens, exposed it in one run. Tightening the schema prompt to require `LIKE '%...%'` for names fixed it, and the same kind of question on the synthetic data ("How many shipments did Orvella Foods import?") now comes back right: 191.

Faithful is not the same as correct

That zero-row answer was judged **faithful**: it accurately reported what its query returned. It was also completely wrong. This is why the project measures grounding and ground-truth correctness as two separate numbers. Anyone reporting a single "accuracy" figure for a RAG system has not hit this yet.

A judge you have not validated is not evidence

If you grade a model with a model, who grades the judge? The harness compares the judge with independent labels on a 12-item subset, each answer read against the evidence it was allowed to use. Labels belong to one run's answers, so after the switch to synthetic data the subset was labelled afresh rather than carried over: 11 of 12 agree, the same as on the first version. The single disagreement is the judge's single miss: on "how many shipments under HS code 1702.1100 (lactose)", the answer had the right count (227) and repeated the question's own "(lactose)" label, which the evidence rows never state. The judge called that unfaithful; the labels call it faithful, because its one load-bearing claim, the count, is exactly the query result. Both are defensible, which is exactly the point.

6 What I verified, and what I did not

The live run was re-done for the synthetic data

When the data was replaced, the live phase was run again once (28 items, about $0.026 of API usage), and the four live tests were run against the API. Every other check below is offline.

- **The gold answers are genuinely ground truth: verified.** They are written by `eval/build_testset.py`, which runs a query for each one, and I re-derived them with independent queries (total USD 274,995,234.91; 2,000 rows; China 208 shipments; top country Brazil; Orvella Foods 191; HS 1704 total 11,665,069.33). Every scope fact checked out too: `trade_type` has exactly one distinct value, `'import'`; dates run 2025-01-01 to 2025-12-31; 28 countries; 28 companies. Two builds of the database are byte-identical.
- **The metrics reproduce: verified.** With credentials unset so no API call was possible, `python -m eval.run_eval --metrics` recomputed every number from the committed transcripts and produced output **byte-identical** to the committed `results.json`.
- **The test suite: verified.** `pytest` with no credentials gives **29 passed, 4 skipped**; with credentials the four live tests pass too, matching the README's claim exactly.

Precisely what "15/15" rests on

All three links are verified for the current numbers: the gold answers are provably correct (re-derived from the database), the scoring provably yields 15/15 (recomputed offline, byte-identical), and the transcripts are the output of a real live run against Gemini, done on 2026-10-14 when the data was replaced.
That run hit Vertex rate limits. Three items waited out `429` responses, so their recorded latencies (27, 46 and 30 seconds) include waiting, which is why p95 is reported both with and without them. The transcripts carry per-call token counts and genuine *failures* (two retrieval misses and one judged unfaithful answer); nothing was re-run to make a number look better.

7 Honest findings

7.1 What the project already owns, and gets right

The wildcard bug and the faithful-but-wrong distinction above are both the project's own findings, documented in its README. So is a third: the first eval run flagged HS-lookup answers as unfaithful because the judge could not see the HS rows the answerer had used. Fixing that plumbing turned the remaining flags into genuine findings. A harness that manufactures failures is as dangerous as one that hides them.

The two remaining faithfulness misses are a single pattern, and it is the best technical observation in the project:

Enumeration is where grounded models leak

In the first version, point-fact answers stayed grounded across the whole set, and "list everything under this heading" is where the model reached past its evidence, padding its list with plausible product names the judge caught by name. A list of three items *feels* incomplete, and the model's training pushes it toward a satisfying list. In the synthetic re-run the lists stayed inside the evidence, most likely because the synthetic HS hierarchy is small enough for the lookup to return every item. That is luck, not a fix. The fix is structural, not a sterner

instruction: post-filter enumerated items against the evidence rows.

7.2 What this review found

Five things worth knowing before an interviewer finds them

1. `in_scope` **is dead code.** The planner is prompted for it, the model reasons about it, the code carefully normalizes it, and *nothing ever reads it.* It costs tokens on every request and reads like a safety control while being none. Either wire it in (it could skip straight to refusal and save two model calls) or delete it.
2. **The SQL guard rejects valid queries.** The keyword regex scans the whole query with no awareness of string literals, so `... WHERE name LIKE '%Create%'` is rejected. I ran this against the real guard to confirm it. A company named "Create Foods" would be unqueryable. It fails *safe*, which is the right direction to err, but the clean fix is SQLite's authorizer callback, which cannot be fooled by quoting.
3. **"Deterministic refusal" is deterministic only in its last step.** All six refusals traced as `['plan', 'refusal']`, meaning no tool ran at all. The guard did not weigh evidence and find it wanting; it fired because the *planner* chose no tools. The decisive judgment belongs to an LLM. 6/6 is real, but the honest claim is narrower than "enforced deterministically".
4. **The 80% retrieval hit rate is really a planner score.** Both misses had no retrieval step in the trace at all: the planner never invoked retrieval, so no chunk could be found. Retrieval went **8 for 8** on everything it was actually asked to find. The metric's name points at the wrong component.
5. **A latent citation bug.** The SQL rows shown to the judge are gated on `used_sql`, a field the *synthesizer reports about itself*, rather than on whether rows exist. That is a structural repeat of the HS-lookup bug already fixed once. I checked all 28 transcripts and found **zero** instances, so the published 95.5% is not contaminated. It is a latent fragility with a one-line fix.

Minor: `jinja2` is pinned in `requirements.txt` and never imported (the server reads its template as a plain string). Per-query costs are real token counts multiplied by *list prices hardcoded in a dict*, not billed amounts, so they will silently drift if Google changes pricing.

7.3 Limits the project states, and should keep stating

- **One run, 28 items.** Every number is a single sample. 15/15 on 15 questions is a good result with wide error bars. "The paths work", not "it is 100% accurate".
- **SQL correctness is scraped from prose**, so a wrong answer containing the right number would score correct. The project documents this itself.
- **The judge and the answerer are the same model family**, so shared blind spots are possible; the independent labels only partly cover this.
- **The corpus is authored summaries, not primary law.** Faithfulness measures grounding in that corpus, not legal accuracy. The project says so in three places and should keep doing so.
- **Latency is high** (p50 about 9 s) because three or four model calls run strictly in sequence, even when the tools are independent.

8 How to defend it

The five things to say

1. **Lead with the eval.** Anyone can wire an LLM to a database. The differentiator is gold answers computed from the database, published metrics including the failures, and a judge checked against independent labels.
2. **Tell the wildcard story.** A fluent answer said 0 shipments; the truth was dozens; the gold number caught it in one run. That is the argument for evaluation in thirty seconds.
3. **Separate faithful from correct**, and note the same bug proved why: the wrong answer was perfectly faithful to its own wrong query.
4. **Defend the hand-built vector store.** 48 chunks, exact search, one matrix-vector product, no server. The simpler thing is also the better thing here.
5. **Volunteer the limits.** 80% is really a routing problem. Enumeration is where grounding leaks. `in_scope` is dead. Say it first.

8.1 Expect these questions

- *"What stops the model dropping your tables?"* Three layers, a test asserts a write fails even with the guard bypassed. Then volunteer the string-literal false positive.
- *"Isn't using an LLM to grade an LLM circular?"* Yes, which is why the judge is checked against independent labels: 11/12, on both the first version and the synthetic re-run, labelled afresh each time.
- *"Faithfulness is 95.5%; why should I trust it?"* Because the judge agrees with independent labels 11 times in 12, and its one miss is a judgment call about a label the user supplied. On the first version it was 86%, and the misses were enumeration padding, a known failure mode with a structural fix.
- *"Would this scale?"* The vector store needs a real index somewhere around six figures of chunks. The tighter constraint is latency: merge the planner with text-to-SQL and parallelise independent tools.

9 Glossary

Agentic retrieval
: Retrieval where a model decides which sources to consult rather than always running one fixed search.

Chunk
: One retrievable piece of a document; here, one section.

Cosine similarity
: How closely two vectors point the same way. With normalized vectors it is just the dot product.

Embedding
: A list of numbers (768 here) representing the meaning of text.

Faithfulness
: Whether an answer is supported by the evidence it was given. Not the same as correctness.

Ground truth
: The known-correct answer, established independently; here, computed straight from the database.

HS code
: International product classification. This dataset is chapter 17, sugar.

LLM-as-judge
: Using a model to grade another model's output.

Precision / recall
: Of the refusals, how many were right / of the questions needing refusal, how many were caught.

RAG
: Fetch the facts, then let the model write using only those facts.

SQLite
: A complete relational database that is a single file, no server.

Text-to-SQL
: Having a model translate a question into a real database query and running it.

Built from the source at `trade-intelligence-copilot`. Every metric quoted was recomputed offline from the committed transcripts, and every gold answer re-derived from the shipped database, before it was written down.